

Раскрашивание изображений с помощью VAE и генеративных сетей

Введение

Цель проекта — разработка модели для автоматического раскрашивания чёрно-белых изображений с использованием вариационного автоэнкодера (VAE).

Проект реализован на языке Python с использованием библиотеки PyTorch и фреймворка ClearML для отслеживания экспериментов. Для начальной проверки архитектуры используется датасет CIFAR-10 (60 000 изображений размером 32×32 пикселя, 10 классов). Было выполнено ознакомление с вариативными автоэнкодерами и их использование для окрашивания черно-белых изображений. Вариационный автоэнкодер — модель, состоящая из двух нейросетей. Одна является кодировщиком, а другая — декодировщиком. Если простыми словами, то первая анализирует данные, а вторая на основе полученного массива выдает нечто похожее.

Создание виртуального окружения

Для проекта было создано отдельное виртуальное окружение с помощью uv, чтобы изолировать зависимости и обеспечить воспроизводимость экспериментов.

Шаги:

- Инициализация проекта `uv init`
- Создание виртуального окружения `uv venv --python 3.10`
- Установка необходимых библиотек через `uv add`:
 - `torch`, `torchvision` — для построения модели
 - `opencv-python` — для конвертации изображений RGB → Lab
 - `numpy`, `matplotlib` — для работы с массивами и визуализации
 - `clearml` — для логирования экспериментов
 - `tqdm` — для прогресс-баров при обучении

Файл `uv.lock` зафиксировал все версии зависимостей для воспроизводимости.

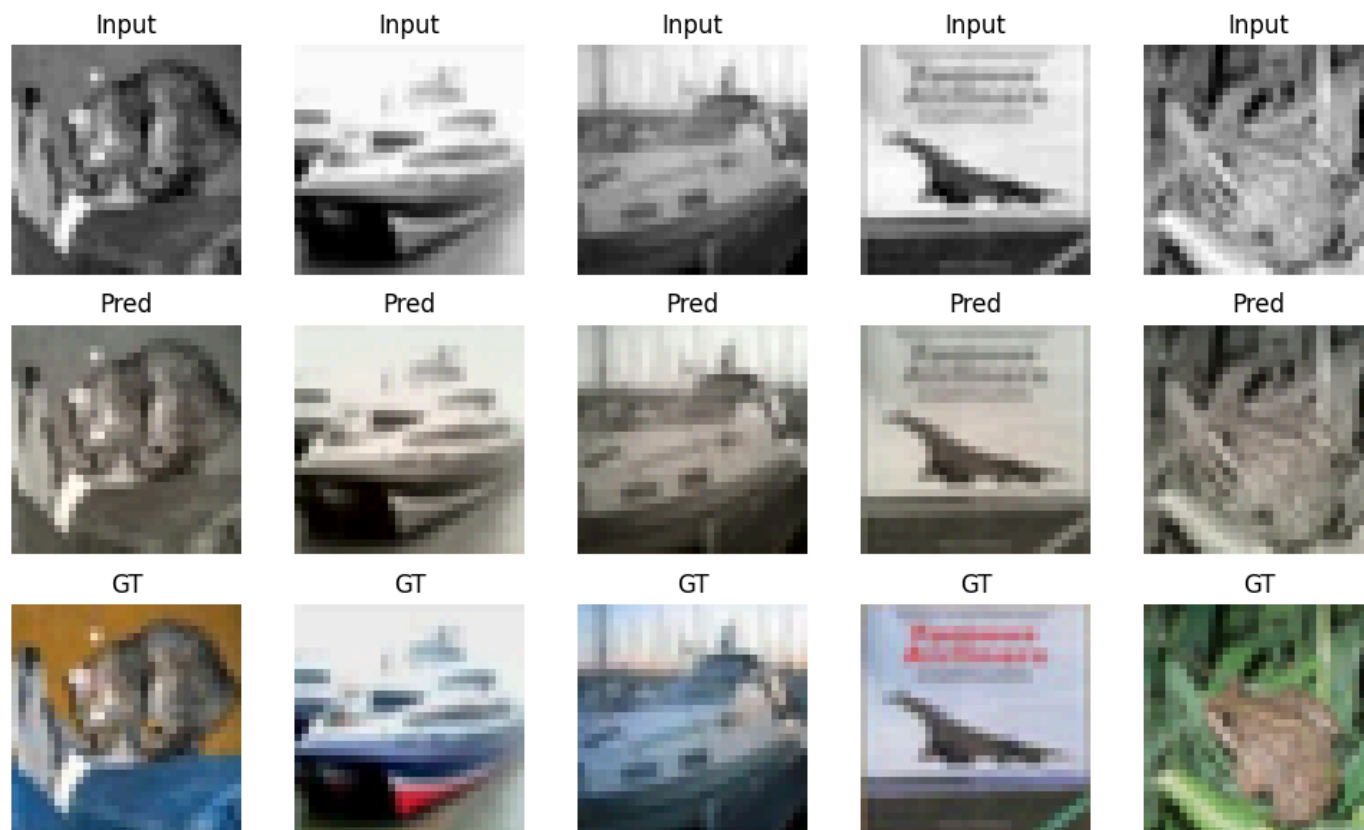
Подготовка датасета

Для загрузки данных был реализован класс `CIFARColorization` в папке `data`:

- Загружает цветные изображения из CIFAR-10 с помощью `torchvision.datasets.CIFAR10`.
- Конвертирует изображения из RGB в цветовое пространство Lab через OpenCV.
- Извлекает:
 - L-канал (яркость) → вход модели (чёрно-белое изображение)
 - ab-каналы (цвет) → цель для обучения
- Выполняет нормализацию:
 - L: 0..1
 - ab: -1..1

Подготовка данных включала преобразование изображений из цветового пространства RGB в LAB. Канал яркости (L) использовался в качестве входа модели, а цветовые каналы (a и b) — в качестве целевых значений для обучения. Такой подход обеспечивает разделение информации о структуре изображения и его цвете, что является стандартной практикой для задач колоризации.

Результат колоризации с помощью VAE:



Как видно по изображениям, автоэнкодера недостаточно для достижения значимых результатов. Поэтому, было принято решение изменить подход и использовать генеративные сети.

Колоризация изображений с использованием GAN

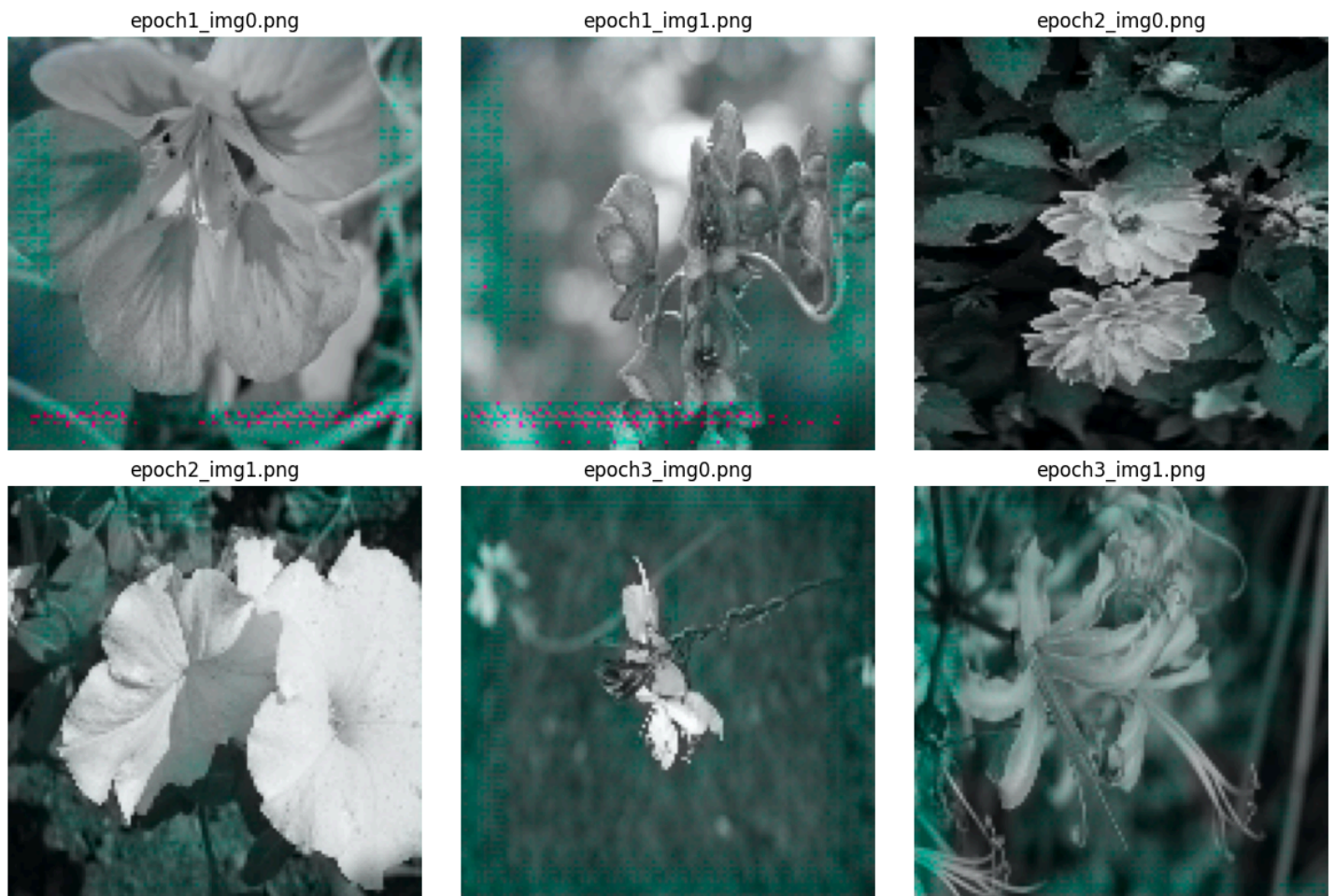
В рамках данной работы была реализована модель для автоматической колоризации черно-белых изображений с использованием генеративно-сопоставительных сетей (GAN). Основная задача заключалась в восстановлении цветовой информации по входному изображению в оттенках серого.

Для обучения использовался датасет Oxford 102 Flowers. Изображения были преобразованы из цветового пространства RGB в LAB. Входом модели служит L-канал, содержащий информацию о яркости, а целевыми значениями являются a и b каналы, отвечающие за цветовую составляющую.

Модель построена на архитектуре Pix2Pix. Генератор реализован в виде U-Net, что позволяет учитывать как глобальные особенности изображения, так и мелкие детали. Дискриминатор основан на PatchGAN и оценивает реалистичность сгенерированных изображений на уровне локальных областей.

К основным ограничениям можно отнести небольшой размер датасета и ограниченное разрешение изображений, что влияет на детализацию. Кроме того, обучение GAN характеризуется чувствительностью к балансу между генератором и дискриминатором.

Результаты первых тестов модели генеративных сетей показаны ниже:



Модель начала давать результаты, корректно раскрашивая траву и листья вокруг цветов в зеленый, но сами лепестки оставляла белыми во всех случаях.

График L1 валидации:



Новая модель была натренирована на большем датасете, COCO (Common Objects in Context). Датасет представляет из себя гораздо более разнообразный набор изображений и объектов, который должен дать гораздо лучшие результаты. Датасет 80 категорий объектов: включает повседневные предметы (например, транспортные средства, бытовая техника), животных, растения и людей. Модель тренировалась на 20000 изображений из этого датасета.

Промежуточный результат:

Input: image.png



Colorized



Original



Input: image2.png



Colorized



Original



Input: image3.png



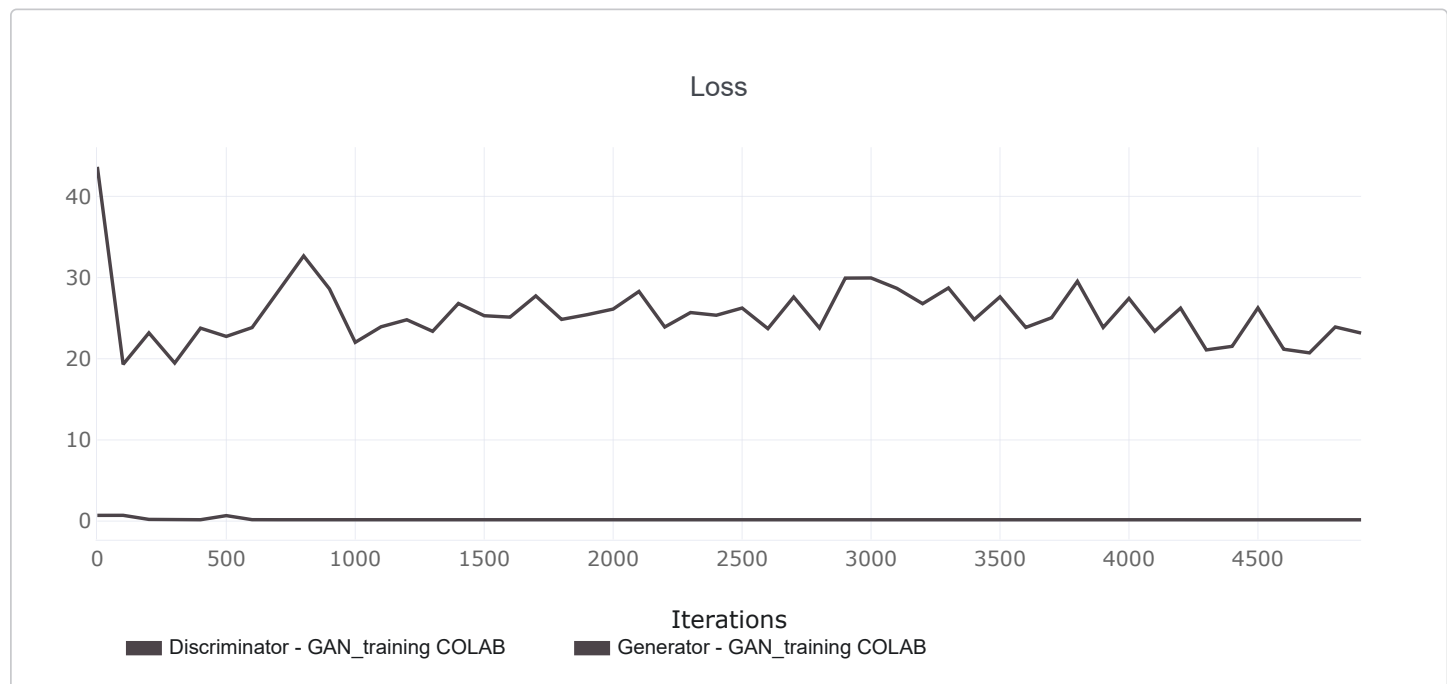
Colorized



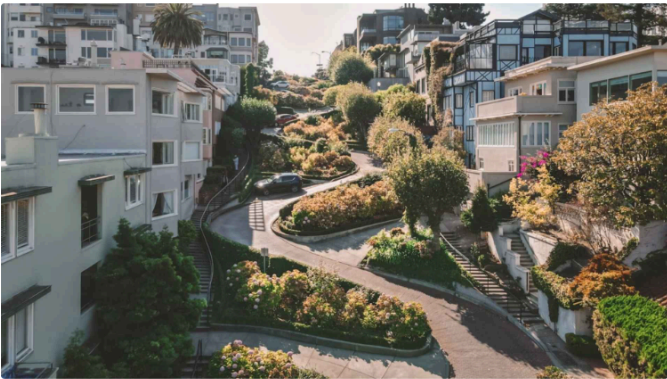
Original



График потери для генератора и дискриминатора модели:



Также было сделано веб-приложение, позволяющее вставить любое изображение для проверки модели. Результаты представлены ниже:

ИСХОДНОЕ (GRAYSCALE) 	РЕЗУЛЬТАТ (COLORIZED) 
ИСХОДНОЕ (GRAYSCALE) 	РЕЗУЛЬТАТ (COLORIZED) 
ИСХОДНОЕ (GRAYSCALE) 	РЕЗУЛЬТАТ (COLORIZED) 

Результаты значительно улучшились и отлично раскрашивают изображение.

График потери для полной модели:



Выводы

Поставленная цель была достигнута, модель действительно генерирует раскрашенные изображения, часто верно определяя цвета, в особенности хорошо справляясь с зеленью и деревьями, а также небом. При этом модель отличает траву от воды, верно генерируя океан синим, что означает что успех модели в идентификации земли/неба зависит не только от позиции пикселей на изображении.

Обучение интегрировано с ClearML, что позволяет отслеживать эксперименты, хранить датасеты и модели централизованно, а также запускать обучение на удалённых машинах через агентов, включая Google Colab с бесплатным GPU. Все метрики, чекпоинты и артефакты доступны в веб-интерфейсе ClearML.

Для тестирования создано веб-приложение на Flask: пользователь загружает изображение, выбирает модель и получает раскрашенный результат в оригинальном разрешении. Приложение работает полностью оффлайн и не требует интернета.

Система решает поставленную задачу — автоматическая колоризация работает, модель выдаёт осмысленные цвета, пайплайн обучения и тестирования автоматизирован.